

AI331: Information Security & Cryptography

Lab 7 — Implementing the Affine Cipher

Instructor: Yugarshi Shashwat

Learning Objectives

By the end of this lab, you will be able to:

- Implement the Affine substitution cipher over the alphabet $\{A, \dots, Z\}$ with correct key validation.
- Compute modular inverses and use them to decrypt affine ciphertexts.
- Perform a basic cryptanalysis of Affine ciphers using two-letter mapping or frequency heuristics.

Pre-reads

Stinson, *Cryptography: Theory and Practice*, Ch. 1–2 (classical ciphers), course notes on modular arithmetic.

1 Affine Cipher: Definition

Let letters map to integers via $A \mapsto 0, B \mapsto 1, \dots, Z \mapsto 25$. For a key (a, b) with $a \in \{1, \dots, 25\}$, $b \in \{0, \dots, 25\}$ and $\gcd(a, 26) = 1$, define:

$$E_{a,b}(x) \equiv (ax + b) \bmod 26, \quad D_{a,b}(y) \equiv a^{-1}(y - b) \bmod 26,$$

where a^{-1} is the multiplicative inverse of a modulo 26.

Valid keys. Since $26 = 2 \cdot 13$, the invertible a 's are

$$\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}.$$

2 I/O & Text Handling Conventions

- Accept plaintext/ciphertext as ASCII strings; ignore characters not in A..Z (or optionally pass them through unchanged).
- Convert letters to uppercase before processing.
- Spaces, punctuation, digits may be removed or preserved (choose and document your choice); tests will follow your documented convention.

3 Task A — Clean Implementation (60 pts)

Goal: Implement encryption and decryption for the Affine cipher with robust key handling.

Requirements

1. **Key validation:** Check $\gcd(a, 26) = 1$ at runtime; reject invalid a with a clear error message.
2. **Modular inverse:** Implement extended Euclidean algorithm to compute $a^{-1} \bmod 26$.
3. **Encrypt/Decrypt:** Functions $E_{a,b}$ and $D_{a,b}$ operating on sanitized text.
4. **CLI interface:**

```
affine --mode enc --a <int> --b <int> --in "<text>"
affine --mode dec --a <int> --b <int> --in "<text>"
```

5. **Tests (include in README or comments):**

- With $(a, b) = (5, 8)$, HELLOWORLD $\rightarrow$ RCLLAOAPLX.
- Decrypting the above must return HELLOWORLD.

Listing 1: AI331 – Lab 7: Affine Cipher (C++ starter skeleton)

```
1  /*
2   AI331      Lab 7: Affine Cipher (C++ starter)
3   -----
4   Build:    g++ -O2 -std=c++17 affine.cpp -o affine
5   Usage:    ./affine enc a b
6             ./affine dec a b
7   Input:    Reads a single line of text from STDIN.
8   Policy:   Keeps only letters A..Z (uppercased). Non-letters are
             dropped.
9  */
10
11 #include <bits/stdc++.h>
12 using namespace std;
13
14 // ----- small helpers -----
15 static inline int mod(int x, int m) {
16     int r = x % m;
17     return (r < 0) ? r + m : r;
18 }
19 static inline bool is_valid_a(int a) {
20     // Valid a iff gcd(a, 26) == 1
21     int x = abs(a), y = 26;
22     while (y) { int t = x % y; x = y; y = t; }
23     return x == 1;
24 }
25
26 // Convert to uppercase A..Z only, drop everything else
27 static string sanitize_letters_only_upper(const string& s) {
28     string t; t.reserve(s.size());
29     for (char c : s) if (isalpha((unsigned char)c)) t.push_back((char)
30         toupper(c));
31     return t;
32 }
33 // ===== \todo: modular inverse a^{-1} (mod 26) via extended Euclid
34 // =====
35 // Return the multiplicative inverse of a modulo m (=26 here).
```

```

35 // If gcd(a, m) != 1, throw.
36 static int modinv(int a, int m) {
37     // \todo: implement extended Euclidean algorithm to find x with a*x
        = 1 (mod m)
38     // Hints:
39     //     - Keep track of (old_r, r), (old_s, s) pairs
40     //     - When loop ends, old_r = gcd(a,m) and old_s is the inverse
        mod m
41     //     - Return mod(old_s, m)
42     throw runtime_error("modinv: TODO");
43 }
44
45 // ===== \todo: encrypt =====
46 // For each letter X in 0..25, compute Y = (a*X + b) mod 26 and map
    back to 'A'+Y.
47 static string encrypt_text(const string& plain, int a, int b) {
48     // \todo: implement  $E_{\{a,b\}}(x) = (a*x + b) \bmod 26$  over sanitized
        uppercase text
49     // Steps:
50     //     - For each character c in 'plain', map  $x = c - 'A'$ 
51     //     -  $y = (a*x + b) \bmod 26$ 
52     //     - push_back('A' + y)
53     // Return the resulting ciphertext.
54     return "TODO_ENCRYPT";
55 }
56
57 // ===== \todo: decrypt =====
58 // Use  $a_{inv} = a^{-1} \bmod 26$ :  $X = a_{inv} * (Y - b) \bmod 26$ .
59 static string decrypt_text(const string& cipher, int a, int b) {
60     // \todo: compute  $a_{inv}$  using modinv(a, 26)
61     // \todo: for each cipher letter  $y = c - 'A'$ , compute  $x = a_{inv} * ($ 
         $y - b) \bmod 26$ 
62     // \todo: map back to 'A' + x
63     return "TODO_DECRYPT";
64 }
65
66 int main(int argc, char** argv) {
67     ios::sync_with_stdio(false);
68     cin.tie(nullptr);
69
70     if (argc != 4) {
71         cerr << "Usage: " << argv[0] << " <enc|dec> a b\n";
72         return 1;
73     }
74
75     string mode = argv[1];
76     int a = stoi(argv[2]);
77     int b = stoi(argv[3]);
78
79     if (!is_valid_a(a)) {
80         cerr << "Error: a must be coprime with 26. Choose a in
        {1,3,5,7,9,11,15,17,19,21,23,25}.\n";
81         return 1;
82     }
83     b = mod(b, 26);
84
85     string line;
86     if (!getline(cin, line)) line = "";

```

```

87
88 // Sanitize: keep only A..Z (uppercased)
89 string in = sanitize_letters_only_upper(line);
90
91 try {
92     if (mode == "enc") {
93         cout << encrypt_text(in, a, b) << "\n";
94     } else if (mode == "dec") {
95         cout << decrypt_text(in, a, b) << "\n";
96     } else {
97         cerr << "Mode must be 'enc' or 'dec'. \n";
98         return 1;
99     }
100 } catch (const exception& e) {
101     cerr << "Error: " << e.what() << "\n";
102     return 1;
103 }
104
105 return 0;
106 }
107
108 /*
109 -----
110 Minimal self-check (manual)
111 -----
112 Example (once TODOs are done):
113
114 $ echo HELLOWORLD | ./affine enc 5 8
115 RCLLAOAPLX
116
117 $ echo RCLLAOAPLX | ./affine dec 5 8
118 HELLOWORLD
119 */

```

4 Task B — Cryptanalysis & Key Recovery (40 pts)

Goal: Recover (a, b) and decrypt given ciphertext(s) produced by an Affine cipher.

B1. Two-letter mapping (deterministic)

If you can guess two plaintext–ciphertext letter correspondences $(x_1 \rightarrow y_1)$ and $(x_2 \rightarrow y_2)$ with $x_1 \neq x_2$, solve

$$y_1 \equiv ax_1 + b \pmod{26},$$

$$y_2 \equiv ax_2 + b \pmod{26}.$$

Subtract to eliminate b : $(y_1 - y_2) \equiv a(x_1 - x_2) \pmod{26}$. If $\gcd(x_1 - x_2, 26) = 1$, then

$$a \equiv (y_1 - y_2) \cdot (x_1 - x_2)^{-1} \pmod{26}, \quad b \equiv y_1 - ax_1 \pmod{26}.$$

B2. Frequency heuristic (practical)

Assume E and T are the most common plaintext letters in English. Let $y_{\max}$ and $y_{2\text{nd}}$ be the two most frequent cipher letters. Try the two pairings:

$$(E \mapsto y_{\max}, T \mapsto y_{2\text{nd}}) \quad \text{and} \quad (E \mapsto y_{2\text{nd}}, T \mapsto y_{\max})$$

Solve for (a, b) via B1 for each pairing; score the resulting decryptions with a dictionary or bigram frequencies, and choose the better one.

Deliverables for Task B.

1. A small script/function that, given ciphertext, attempts the frequency method above and prints candidate keys & plaintext.
2. A short writeup (5–10 lines) describing which strategy succeeded and why.

5 Test Vectors

Key (a, b)	Plaintext	Ciphertext
$(5, 8)$	HELLOWORLD	RCLLAOAPLX
$(7, 3)$	CRYPTOGRAPHY	RSPEGXTSDEAP

6 Edge Cases & Notes

- If you choose to *preserve* non-letters, ensure decryption restores them in place.
- Reject invalid a immediately; do not attempt decryption without a^{-1} .
- Document your sanitization policy at the top of your source file(s).

7 What To Submit

1. README.md: build/run instructions; text-handling policy; brief summary of Task B approach.
2. Source files: `affine.cpp`
3. Optional: small test script and sample outputs.

Academic Integrity: Write your own code. Cite any external references used.

8 Background: Extended Euclidean Algorithm

To decrypt in the Affine cipher, we need the multiplicative inverse $a^{-1} \bmod 26$. This requires solving the congruence

$$a \cdot a^{-1} \equiv 1 \pmod{26}.$$

The method to compute such inverses is the **Extended Euclidean Algorithm** (EEA).

The Euclidean Algorithm

Recall that for integers a, b with $a > b > 0$, the Euclidean algorithm finds $\gcd(a, b)$ by repeated division:

$$a = q_1b + r_1, \quad b = q_2r_1 + r_2, \quad r_1 = q_3r_2 + r_3, \quad \dots$$

until some remainder is 0. The last nonzero remainder is $\gcd(a, b)$.

Extended version

The extended form not only computes $\gcd(a, b)$, but also integers x, y such that

$$ax + by = \gcd(a, b).$$

This is called a *Bézout identity*. If $\gcd(a, b) = 1$, then x is precisely the inverse of $a \pmod{b}$.

Algorithm sketch

1. Initialize $(\text{old_r}, r) = (a, b)$, $(\text{old_s}, s) = (1, 0)$, $(\text{old_t}, t) = (0, 1)$.

2. While $r \neq 0$:

$$q = \lfloor \text{old_r} / r \rfloor, \quad (\text{old_r}, r) \leftarrow (r, \text{old_r} - q \cdot r),$$

and update s, t similarly:

$$(\text{old_s}, s) \leftarrow (s, \text{old_s} - q \cdot s), \quad (\text{old_t}, t) \leftarrow (t, \text{old_t} - q \cdot t).$$

3. When the loop ends, $\text{old_r} = \gcd(a, b)$, and $(\text{old_s}, \text{old_t})$ satisfy $a \cdot \text{old_s} + b \cdot \text{old_t} = \gcd(a, b)$.

Worked example

Find a^{-1} for $a = 5 \pmod{26}$.

$$26 = 5 \cdot 5 + 1.$$

So $1 = 26 - 5 \cdot 5$. Rearrange: $1 = (-5) \cdot 5 + 1 \cdot 26$.

Thus $x = -5$ is a solution: $5 \cdot (-5) \equiv 1 \pmod{26}$. Taking modulo 26, we get $a^{-1} = 21$.

Check: $5 \cdot 21 = 105 \equiv 1 \pmod{26}$.

Hence, the extended Euclidean algorithm lets us compute the inverse a^{-1} efficiently for any valid a in the Affine cipher.